

Data mining & machine learning

1. Hyperparameter and parameter optimization

Francesco Pistolesi

University of Pisa

`francesco.pistolesi@unipi.it`

M.Sc. in Artificial Intelligence and Data Engineering



UNIVERSITÀ DI PISA



Francesco Pistolesi

University of Pisa

Department of Information Engineering

Office Hours

Thursday, 16:30 – 18:30

Office: Building A, 2nd Floor

Department of Information Engineering

Please send an email in advance
to schedule an appointment.



Building A: It looks taller from the outside, but inside there are only two floors: don't ask why!

In this course we will cover:

- 1 How to **optimize** ML models
- 2 How to design & train a **regressor**
- 3 How to predict values from **time series**
- 4 How to detect **outliers**
- 5 How to deal with text (**text mining**)



If you are attending this second part of the course, you need to get 12 credits (CFU) and complete the whole course (Basic + Advanced). The final exam for 12 CFU consists of:

- A **project** that ... *this pause creates hype*
- An **oral exam** on all the topics covered by the *theoretical part* of the course

You have **3 exam dates per session**. The final score considers the project and the oral exam.

In a given date, you can discuss the project, take the oral exam, or both. But with **some rules**.

Exam Rules

If you decide to take the exam on a specific date, please follow these rules.

You must

- **Register** on the *Valutami portal*. Registrations close **72 hours before the exam date** (no exception). In case of problems with the portal, send an email before the deadline.
- **Specify which part(s)** you will take, in the Note field by typing one option among:
 - Only project
 - Only oral
 - Project+Oral

You cannot

- **Take the oral exam before discussing the project** with a positive outcome

You can

- Discuss the project on an exam date and **take the oral on a later date**.
- **Repeat the oral exam** due to rejection or unsatisfactory outcome: the **project remains valid**.

The 72-hour deadline is required to properly organize the exam.

The final project involves at least one task among *classification*, *regression*, *forecasting*, *mining association rules*, *mining text*, etc.

The preliminary steps are as follows:

- Find a **classmate** to work with (projects should be **in pairs**).
- Look for a **dataset** (e.g., using repositories such as UCI, Kaggle, OpenML, etc.)
- Decide a **ML task**
- Prepare a **one-page proposal**
- Submit your proposal via the course **Elearn page** (available soon).
- **Wait** for approval
- **Start working** to the project

Structure of the proposal

The proposal is **limited to 1 page** and must have the structure as follows:

- 1 Domain and core idea
- 2 Dataset
- 3 ML tasks performed
- 4 Motivation

You can submit the proposal **via the course Elearn page** (available soon). You will find a **template for the project proposal**.



The project submission must include:

- ❶ A **written report** (maximum 10 pages)
 - Download the **template** and maintain section titles, style, and structure.
 - Less is more: **be very specific** and **discuss pros and cons** of your approach.
 - Use **simple language** and **short sentences**.
 - Number figures and tables, insert captions, and reference figures and tables in the text.
 - **Do no exceed the page limit.** Do not modify margins or font size!
- ❷ The **source code**
 - You have to submit a **zip file** with all the source code.
- ❸ A **video presentation** (maximum 10 minutes)
 - *PowerPoint* guide: <https://tinyurl.com/3ps79v6k>
 - *Apple Keynote* guide
<https://support.apple.com/en-au/guide/keynote/tan81813d552/mac>
 - External tools (e.g., *OBS studio* for Windows, macOS, and Linux)

- 1 Correctness, completeness, and rigor
- 2 Clarity of the presentation (report and video)
- 3 Answers to questions during the project discussion
(in person, on the exam date you register for Only Project or Project+Oral)

By the end of the lecture, you will be able to:

- distinguish **parameters** from **hyperparameters**;
- explain why hyperparameter tuning is a **model selection** problem;
- use GridSearchCV and RandomizedSearchCV;
- tune complete **pipelines**;
- understand why **nested cross-validation** gives a less biased performance estimate.

Main topics

parameters vs hyperparameters

grid vs random search

pipeline tuning

nested CV

Notebook structure

- 1 Parameters and hyperparameters
- 2 Hyperparameter tuning
- 3 Grid search vs random search
- 4 Pipelines and composite estimators
- 5 Nested cross-validation
- 6 Practical considerations and further reading

understand the problem

search efficiently

tune the whole workflow

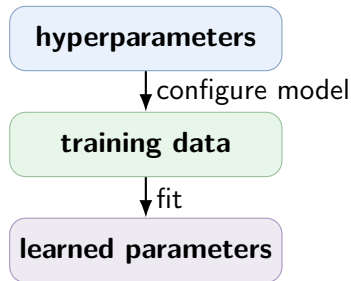
evaluate fairly

Parameters vs hyperparameters

Most machine learning algorithms are based on a form of **multi-level inference**.

A model is defined by:

- a set of **parameters** learned from data;
- a set of **hyperparameters** chosen before training.



Decision tree example

- Parameters: attributes and split points selected at internal nodes.
- Hyperparameters: splitting criterion, maximum depth, minimum samples per split, and so on.

In scikit-learn, hyperparameters are constructor arguments

```
from sklearn.tree import DecisionTreeClassifier

DecisionTreeClassifier(
    criterion='gini',
    splitter='best',
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features=None,
    random_state=None,
    ccp_alpha=0.0
)
```

Read this slide this way

- Constructor arguments = **hyperparameters**
- `fit(X,y)` learns the **parameters**
- The code already tells us what can be tuned

A compact comparison

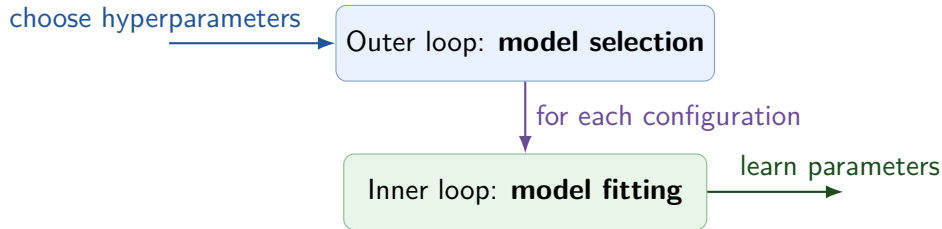
Aspect	Parameters	Hyperparameters
Who sets them?	learned automatically during training	chosen before training
Where do they appear?	inside the fitted model	in the estimator constructor
Examples	weights, split thresholds, support vectors	C, max_depth, gamma
Typical objective	fit the training data	optimize generalization performance

inner optimization

outer optimization

Why this distinction is useful

This separation is mainly for **computational convenience**.

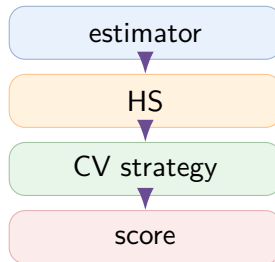


- Outer loop: adjust hyperparameters to optimize a model selection criterion.
- Inner loop: fit model parameters on the training data.

Hyperparameter tuning as a search problem

From the scikit-learn documentation, a search consists of:

- an **estimator**;
- a **hyperparameter space (HS)**;
- a method for **searching or sampling** candidates;
- a **cross-validation** scheme;
- a **score function**.



In short

We must explore a search space and estimate generalization performance for each candidate.

Running example used in the notebook

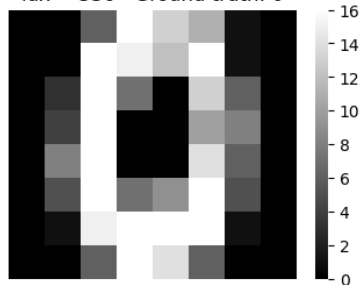
Notebook setup

- dataset: `load_digits(..., n_class=3)`
- classifier: decision tree with `random_state=42`
- validation: `StratifiedKfold(n_splits=10)`
- default score: **accuracy**

The search space will have two hyperparameters:

- `max_depth`
- `min_samples_split`

idx = 536 - Ground truth: 0



small example

fast to visualize

The stage is set: estimator, CV, score, parameter space

```
from sklearn.datasets import load_digits
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold

X, y = load_digits(return_X_y=True, n_class=3)
skf = StratifiedKFold(n_splits=10)
clf = DecisionTreeClassifier(random_state=42)

param_space = {
    "max_depth": range(2, 15),
    "min_samples_split": range(2, 10, 2),
}
```

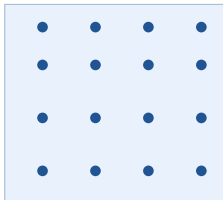
Search ingredients

- estimator = clf
- CV = skf
- score = accuracy
- space = 2D grid

When the number of hyperparameters is small

Grid search

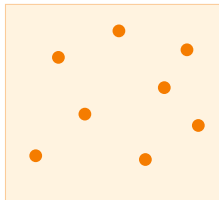
Exhaustively evaluate all combinations in a predefined grid.



structured coverage

Random search

Sample a fixed number of candidates from the parameter space according to a distribution.



diverse coverage

```
from sklearn.model_selection import
    RandomizedSearchCV

random_search = RandomizedSearchCV(
    clf,
    param_distributions={
        "max_depth": range(2, 15),
        "min_samples_split": range(2, 10, 2),
    },
    n_iter=15,
    random_state=42,
    cv=skf,
)

random_search.fit(X, y)
```

Idea

Spend a fixed budget of trials and let randomness explore the space.

- 15 candidate settings
- about 0.35 seconds
- best mean CV accuracy: about **0.940**

Top-ranked sampled settings include:

- `max_depth = 12, min_samples_split = 4`
- `max_depth = 13, min_samples_split = 8`
- `max_depth = 14, min_samples_split = 6`

Interpretation

Even with a limited budget, random search already visits very competitive regions of the search space.

```
from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(
    clf,
    param_grid={
        "max_depth": range(2, 15),
        "min_samples_split": range(2, 10, 2),
    },
    cv=skf,
)

grid_search.fit(X, y)
```

Idea

Map the full grid. Good for small spaces and for understanding the landscape.

- 52 candidate settings
- best mean CV accuracy: about **0.940**

One best setting reported:

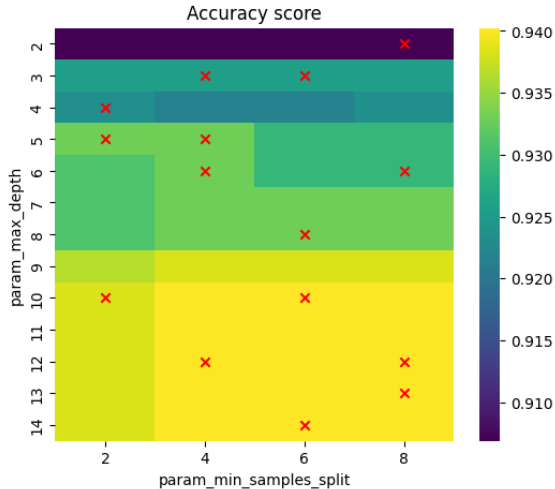
- `max_depth = 10, min_samples_split = 4`

Several neighboring settings share the same best rank.

Key message

Here grid search spends more evaluations, but it does *not* improve the best score compared with the random-search run in the notebook.

Visual comparison on the decision tree search space



How to read it

- color = mean CV accuracy
- red crosses = sampled random-search points

What do we learn from the heatmap?

- There is a broad plateau of good solutions for deeper trees.
- The best region is *not* a single isolated point.
- Random search hits competitive settings with far fewer evaluations.
- Grid search gives a complete map, but that complete map has a cost.



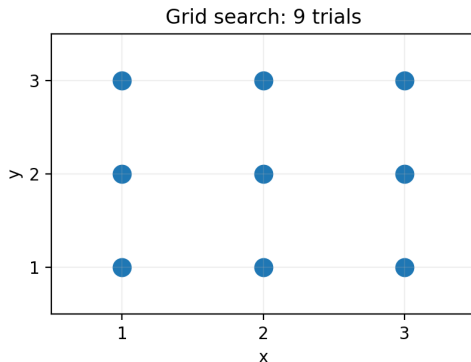
Low effective dimensionality

Hyperparameter optimization problems are often **much more sensitive to some dimensions** than to others.

If we already know the influential dimensions, we can design a very good grid.

If we *do not* know them in advance, **random search is usually safer and more efficient.**

Why random search can be better with the same budget



Suppose $x, y \in \{1, 2, 3\}$ are two hyperparameters. With **9 trials**, random search covers **more values along each axis**. This is key when one dimension is much more important than the other.

A useful abstraction: low effective dimensionality

Suppose that the validation performance can be written as follows:

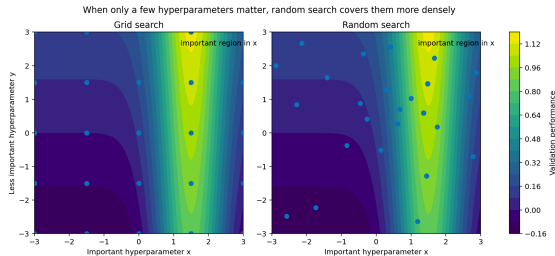
$$f(x, y) = g(x) + \varepsilon(y), \quad |\varepsilon(y)| \ll |g(x)|.$$

Equivalently,

$$\left| \frac{\partial f}{\partial x} \right| \gg \left| \frac{\partial f}{\partial y} \right|.$$

Then:

- performance changes strongly along x ;
- performance changes weakly along y ;
- a budget of N random trials explores more distinct values of the important variable x than a $\sqrt{N} \times \sqrt{N}$ grid.



Same budget, same search space: random search covers the important region in x more densely.

The notebook we will see briefly mentions more sophisticated methods, including:

- **Bayesian optimization**
- **gradient-based** approaches

Remember

Grid and random search are excellent baselines, but they are not the only options.

A practical rule of thumb

Do not tune blindly

A search algorithm can find good configurations, but it **cannot replace model understanding**. Always inspect the **meaning**, **scale** and **expected effect** of the hyperparameters you tune.

search algorithm

+

model understanding

A practical workflow for hyperparameter optimization

Formal problem

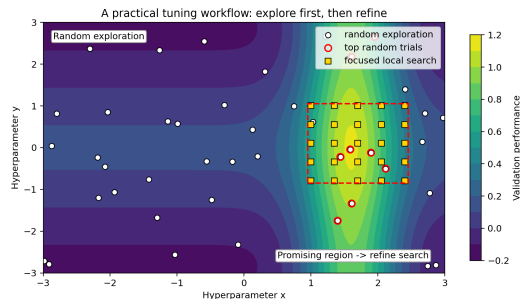
$$\lambda^* = \arg \max_{\lambda \in \Lambda} f(\lambda)$$

where:

- $\lambda \in \Lambda \subseteq \mathbb{R}^d$ is a vector of hyperparameters;
- Λ is the **search space** (domain of hyperparameter configurations);
- $f(\lambda)$ is the validation score (e.g., accuracy, F1-score, AUC, RMSE)

Why not just use a large grid?

- $f(\lambda)$ is a **black-box** function whose structure is unknown before the search;
- each evaluation is computationally expensive.



Practical implication

- ① start with a broad **random exploration**;
- ② **inspect** the results;
- ③ **restrict** ranges and **refine** the search.

What should you remember about grid vs random search?

The key message

Random search is **not universally better** than grid search. It is often a better *default choice* when we do **not know in advance** which hyperparameters are more influential.

- If all hyperparameters are equally important and the search space is tiny, grid search is fine.
- If only a few hyperparameters dominate performance, random search is usually more efficient.
- In real projects, we often use **random search first**, then a **more focused local search**.

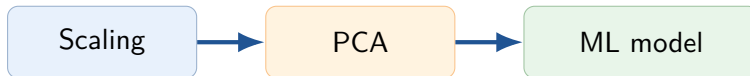
explore

analyze

refine

Putting it all together: tuning complete pipelines

Real workflows rarely contain just one estimator.



Key point

We may want to tune the whole pipeline *jointly*, not each step independently.

Why tune the whole pipeline?

- ① **Steps interact with each other:** *preprocessing* and *feature engineering* affect what the model sees and its behavior. Optimizing each step separately may lead to **suboptimal** combinations.
- ② **We want to evaluate the full process:** the validation score should reflect the performance of the **entire pipeline**, not isolated components.
- ③ **Avoid data leakage:** preprocessing **must only be fitted on training data** within each split. A unified pipeline ensures this happens correctly during cross-validation.

Something to remember

Treat the pipeline as a **single system** and tune it end-to-end.

Pipeline example from the notebook

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA()),
    ("logistic", LogisticRegression()),
])
```

Important syntax

- `pca__n_components`
- `logistic__C`

Parameters of nested estimators use the `<step>__<parameter>` convention.

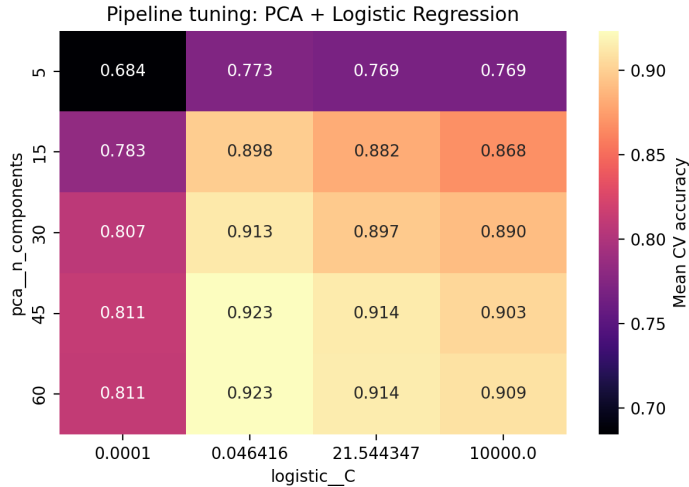
Searching over pipeline hyperparameters

```
param_grid = {  
    "pca__n_components": [5, 15, 30, 45, 60],  
    "logistic__C": np.logspace(-4, 4, 4),  
}  
  
search = GridSearchCV(pipe, param_grid, cv=skf,  
    n_jobs=-1)  
search.fit(X_digits, y_digits)
```

Notebook result

- best CV score $\approx$ **0.923**
- best parameters:
 - `pca__n_components` = 60
 - `logistic__C` = 0.0464

Pipeline search results

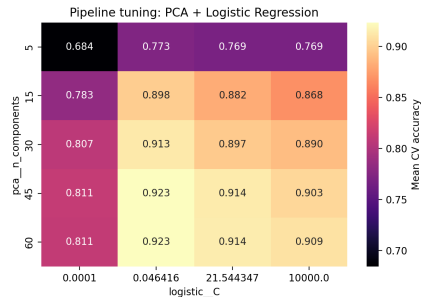


Reading guide

rows = retained components
columns = regularization strength

Interpretation of the pipeline heatmap

- Too few principal components hurt accuracy.
- The regularization strength C also matters.
- The best region in the notebook is around:
 - many retained components;
 - moderate regularization.
- Pipeline tuning lets preprocessing and classifier be optimized **together**.

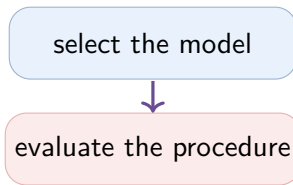


Model selection vs. performance evaluation

There are two core components of good ML practice:

- 1 **Model selection:** choose algorithms and hyperparameters.
- 2 **Performance evaluation:** estimate generalization capability.

Hyperparameter tuning belongs to model selection.



Question

How do we evaluate **fairly and without bias**?

Question A: can we trust the best CV score from tuning?

Tempting but wrong

If we use `GridSearchCV` or `RandomizedSearchCV`, is the score for the best setting automatically a good estimate of generalization performance?

No!

Why a single CV level can be optimistic

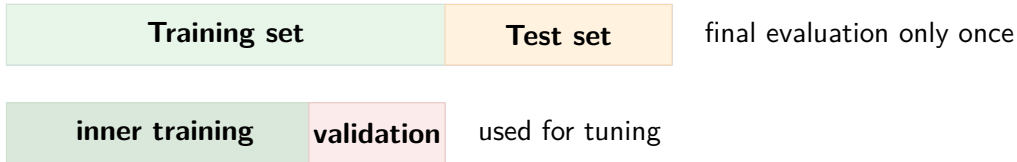
When we evaluate many hyperparameter settings:

- we keep adjusting the model until it performs best on the validation folds;
- information from those folds can effectively leak into the selection process;
- the selected score becomes **optimistically biased**.

Core issue

The same resampling procedure should not safely play both roles at once: selecting the model and evaluating its final generalization performance.

Solution B: if a hold-out test set is available



Tune on the training data, then evaluate only once on the untouched test set.

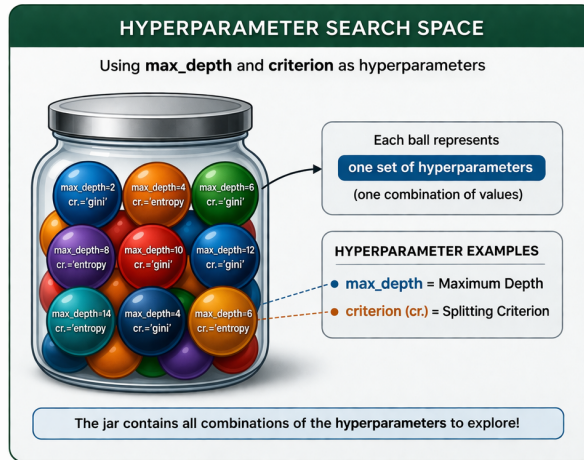
Solution C: if no hold-out test set is available

Nested cross-validation

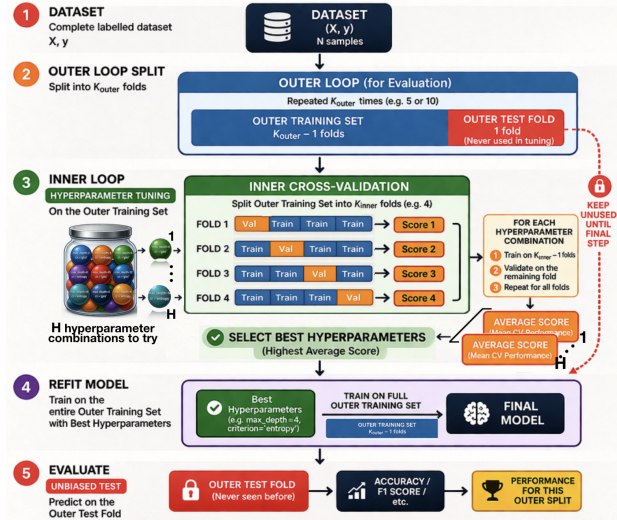
Use one cross-validation loop for model selection and another, outer loop for performance evaluation.

- **Inner CV:** choose hyperparameters.
- **Outer CV:** estimate the generalization error of the whole selection procedure.

Hyperparameter search space

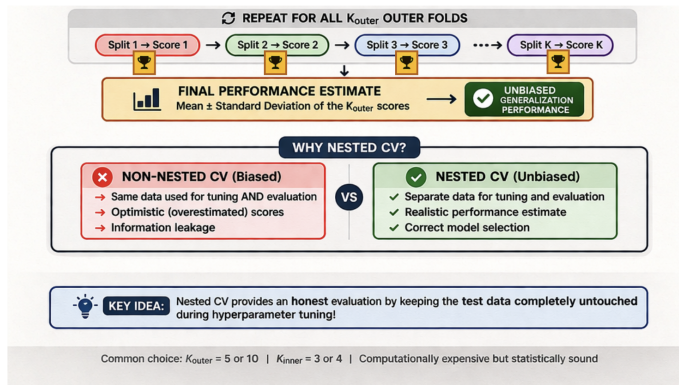


Nested CV at a glance



IMPORTANT: In this slide, “hyperparameters” is short for “hyperparameter combination”, i.e., one combination of hyperparameter values among those selected during the preliminary sampling phase based on Grid search, Random search, etc.

Nested CV at a glance (cont.)



Nested Cross-Validation: Complete Algorithm

Let K^{OUT} be the number of outer folds, K^{IN} the number of inner folds, and $\mathcal{H} = \{h_j\}_{j=1}^H$ the set of combinations of hyperparameter values previously selected (e.g., via Grid sampling).

Procedure

For each outer fold $o = 1, \dots, K^{OUT}$:

- ① Let fold o be the *outer test set*, and the other folds form the *outer training set*.
- ② Divide the outer training set into K^{IN} folds.
- ③ **For each** combination of hyperparameter values $h_j \in \mathcal{H}$:
 - ① Perform a K^{IN} -fold cross validation instantiating models based on h_j ;
 - ② Calculate the average performance $p_{o,j}^{IN}$ on the K^{IN} inner validation folds.
- ④ Select the best hyperparameters, i.e., the combination $h_{o,j}^*$ with the highest $p_{o,j}^{IN}$.
- ⑤ Train the model configured using $h_{o,j}^*$ on the entire outer training set.
- ⑥ Evaluate the performance p_{o,j^*}^{OUT} of the resulting model on the outer test set.

Final performance estimate: $\bar{p} = \frac{1}{K^{OUT}} \sum_{o=1}^{K^{OUT}} p_{o,j^*}^{OUT}$; $\sigma = \sqrt{\frac{1}{K^{OUT}-1} \sum_{o=1}^{K^{OUT}} (p_{o,j^*}^{OUT} - \bar{p})^2}$.

A Python implementation

```
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC

# Define hyperparameter search space
param_grid = {
    "C": [0.1, 1, 10],
    "gamma": [0.001, 0.01],
    "kernel": ["rbf"]
}

# Inner loop: hyperparameter optimization
grid_search = GridSearchCV(
    estimator=SVC(),
    param_grid=param_grid,
    cv=5          # inner CV
)

# Outer loop: performance estimation
scores = cross_val_score(grid_search, X, y, cv=5)
print("Nested CV accuracy:", scores.mean())
```

What Happens to Hyperparameters Across Folds?

Each outer fold may produce a different best configuration:

Outer Fold	Best Hyperparameter
1	$C = 1$
2	$C = 10$
3	$C = 1$
4	$C = 0.1$
5	$C = 1$

This is expected because each training subset is different.

Nested CV estimates the performance of the **model selection procedure**.

Final Training

After nested CV:

- 1 Use the entire dataset
- 2 Run hyperparameter search again
- 3 Select the best configuration
- 4 Train the final model

This final model is used for future predictions.

The notebook's nested-CV experiment

```
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV,
    KFold, cross_val_score

p_grid = {"C": [1, 10, 100],
          "gamma": [0.01, 0.1],
          "kernel": ["rbf"]}

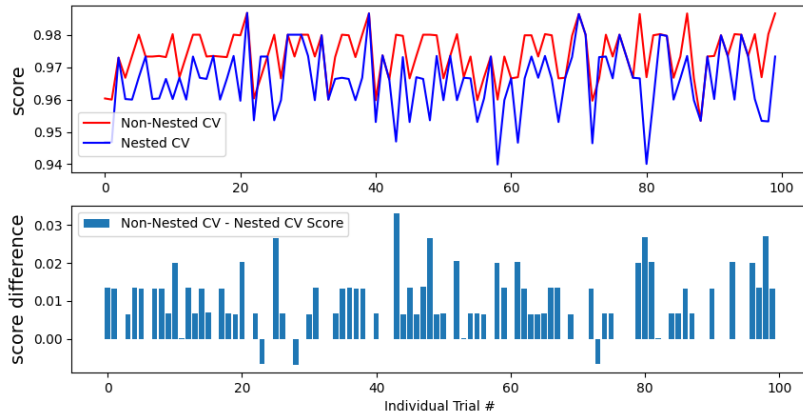
inner_cv = KFold(n_splits=4, shuffle=True,
                 random_state=i)
outer_cv = KFold(n_splits=4, shuffle=True,
                 random_state=i)
clf = GridSearchCV(SVC(), param_grid=p_grid, cv=
                  inner_cv)
```

Setup

Dataset used in the notebook:
load_iris() with NUM_TRIALS
= 100.

Nested vs non-nested CV results from the notebook

Non-Nested and Nested Cross Validation on Iris Dataset



Observed gap

average score difference

$\approx$ **0.0082**

standard deviation $\approx$

0.0082

What should you remember about nested CV?

- **Non-nested CV** often looks slightly better because it **reuses the same data** for selection and evaluation.
- **Nested CV** estimates the performance of the *full model-selection procedure*, not just one fitted model.
- It is more **computationally expensive**, but returns **realistic performance**.

less bias

more computation

Practical considerations from the notebook

- Nested CV is substantially more expensive than flat CV.
- The notebook also cites a 2021 paper arguing that nested CV may be *overzealous* in many practical settings.
- For algorithms with relatively few hyperparameters, a cheaper flat-CV procedure may often select a classifier of very similar practical quality.

Nuanced message

Nested CV is the safest methodological choice, but computational cost and practical impact both matter.

- 1 Understand the model before tuning it.
- 2 Start with a small, meaningful hyperparameter space.
- 3 Use random search when the space is large or poorly understood.
- 4 Tune whole pipelines when preprocessing affects the classifier.
- 5 Separate model selection from final evaluation.
- 6 Use nested CV when an unbiased estimate is essential and no hold-out test set exists.

- Bergstra, J. and Bengio, Y. (2012), *Random Search for Hyper-Parameter Optimization*.
- Varma, S. and Simon, R. (2006), *Bias in Error Estimation When Using Cross-Validation for Model Selection*.
- Wainer, J. and Cawley, G. (2021), *Nested cross-validation when selecting classifiers is overzealous for most practical applications*.
- Scikit-learn: multi-metric evaluation with `cross_val_score` and `GridSearchCV`.
- Scikit-learn: successive halving for hyperparameter search.

Core ideas

- Parameters are learned; hyperparameters are chosen.
- Hyperparameter tuning is a **search problem**.
- Grid search is exhaustive but can be wasteful.

Methodological message

- Random search is often more efficient.
- Pipelines optimize preprocessing and classification jointly.
- Nested CV reduces selection bias in performance evaluation.

understand the model

search smartly

evaluate fairly